

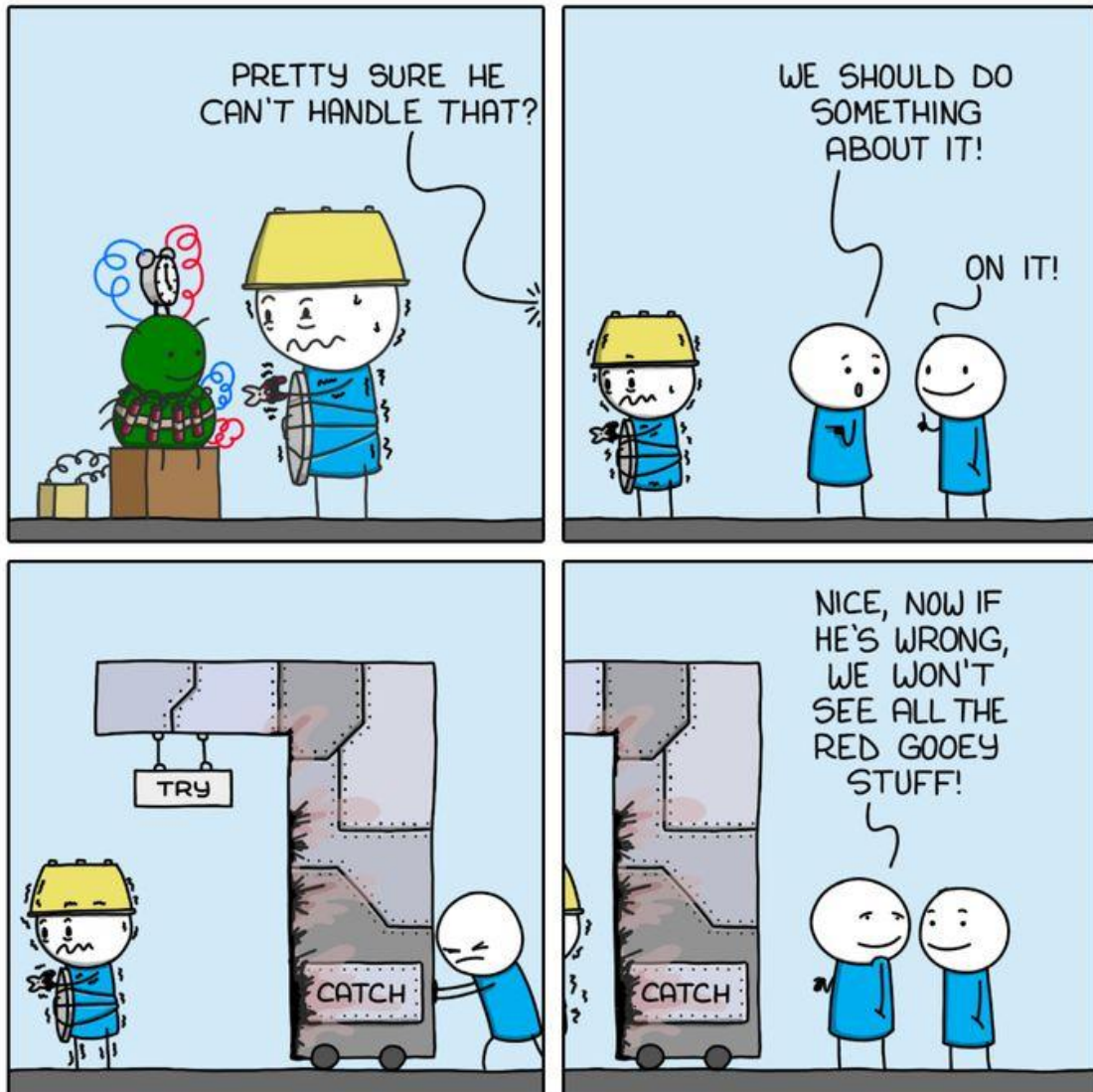
Modul 6

Exception Handling

Praktikum Pemrograman Berorientasi Objek 2025/2026

ANTICIPATE

MONKEYUSER.COM



Universitas Atma Jaya Yogyakarta

Fakultas Teknologi Industri

Program Studi Informatika

Tujuan Pembelajaran

1. Memahami konsep **Exception Handling** dan cara mengimplementasikannya dalam Java.
2. Dapat membuat **User-Defined Exception** untuk menangani kasus spesifik dalam program.

Pendahuluan

Dalam pemrograman, kita tidak dapat menghindari kesalahan yang disebut sebagai Error. Error merupakan bagian yang tidak terpisahkan dari proses pengembangan perangkat lunak. Kesalahan bisa terjadi karena berbagai faktor, seperti kesalahan dalam penulisan kode atau kesalahan logika yang menyebabkan hasil yang tidak sesuai dengan yang diharapkan. Kesalahan dalam pemrograman dapat menyebabkan program tidak dapat berjalan dengan baik, menampilkan hasil yang salah, atau bahkan mengalami crash. Secara umum, error dalam pemrograman diklasifikasikan menjadi tiga, yaitu:

1. Syntax Error

Error ini dapat terjadi ketika programmer menggunakan syntax yang salah saat menulis code, sehingga menyebabkan error pada waktu kompilasi. Ini biasanya dapat dihindari dengan fitur netbeans, yang jika terjadi syntax error muncul underline merah dan hint perbaikannya. **Contoh:**

- Lupa titik koma (;)
- Salah ketik huruf (string dengan 's' kecil, throws tanpa s)

2. Semantic Error

Error ini terjadi ketika logika program salah, sehingga output tidak seperti dengan apa yang diharapkan. **Contoh:**

- Rumus yang salah (misalnya luas persegi panjang = panjang + lebar),
- Jumlah loop yang salah (misalnya tertulis n-1, seharusnya n+1),
- Kondisi yang salah (misalnya if x%2==0 berarti bilangan ganjil, seharusnya genap)

3. Runtime Error

Runtime Error baru terdeteksi saat eksekusi program. Error ini dapat menyebabkan program berhenti secara tidak normal yang biasa disebut crash. **Contoh:**

- Pembagian dengan 0.
- Menggunakan indeks array yang lebih besar dari ukuran array.
- Ketika program meminta anda untuk menginput angka, tetapi anda menginputkan huruf.

Exception adalah kondisi yang disebabkan oleh **Runtime Error**. Agar program tetap berjalan normal, diperlukan mekanisme penanganan (handling) yang disebut **Exception Handling** untuk menangani error sebagai berikut:

1. **Try:** Blok untuk menuliskan kode yang berpotensi menyebabkan error.
2. **Catch:** Blok untuk menangani exception yang terjadi di dalam blok try.
3. **Finally:** Blok yang akan selalu dieksekusi, baik terjadi exception maupun tidak.
4. **Throw:** Digunakan untuk melemparkan objek exception secara eksplisit.
5. **Throws:** Digunakan pada deklarasi method atau konstruktor untuk menunjukkan bahwa method tersebut mungkin menghasilkan exception.

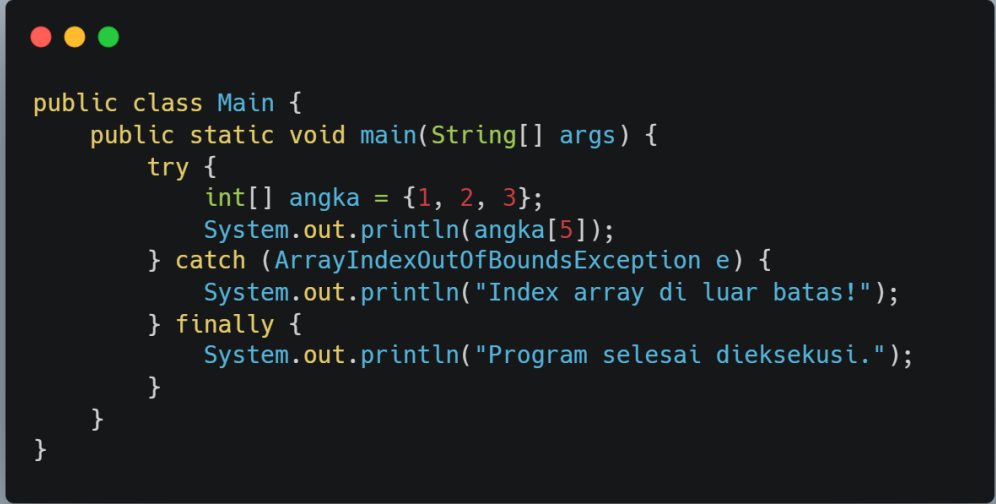
Berikut adalah bentuk dari try, catch, finally :

```
try {  
    // berisikan code-code yang berpotensi menyebabkan exception  
} catch (Exception e1) {  
    // berisikan kode yang digunakan untuk menangani exception ke-1  
} catch (Exception eN) {  
    // berisikan kode yang digunakan untuk menangani exception ke-N  
} finally {  
    // blok ini akan selalu jalan setelah catch-catch sudah terlewati  
}
```

Mekanisme Try-Catch-Finally :

1. Blok try berisi kode yang berpotensi menimbulkan exception.
2. Jika terjadi exception, program mencari blok *catch* yang sesuai untuk menangani error.
3. Jika tidak ada *catch* yang cocok, exception ditangkap oleh blok catch yang lebih general.
4. Jika tidak ada *catch* sama sekali, program akan crash.
5. Blok finally selalu dijalankan, baik terjadi exception atau tidak, untuk memastikan cleanup atau proses akhir tetap berjalan.

Contoh Penerapan Try-Catch-Finally :



```
public class Main {  
    public static void main(String[] args) {  
        try {  
            int[] angka = {1, 2, 3};  
            System.out.println(angka[5]);  
        } catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("Index array di luar batas!");  
        } finally {  
            System.out.println("Program selesai dieksekusi.");  
        }  
    }  
}
```

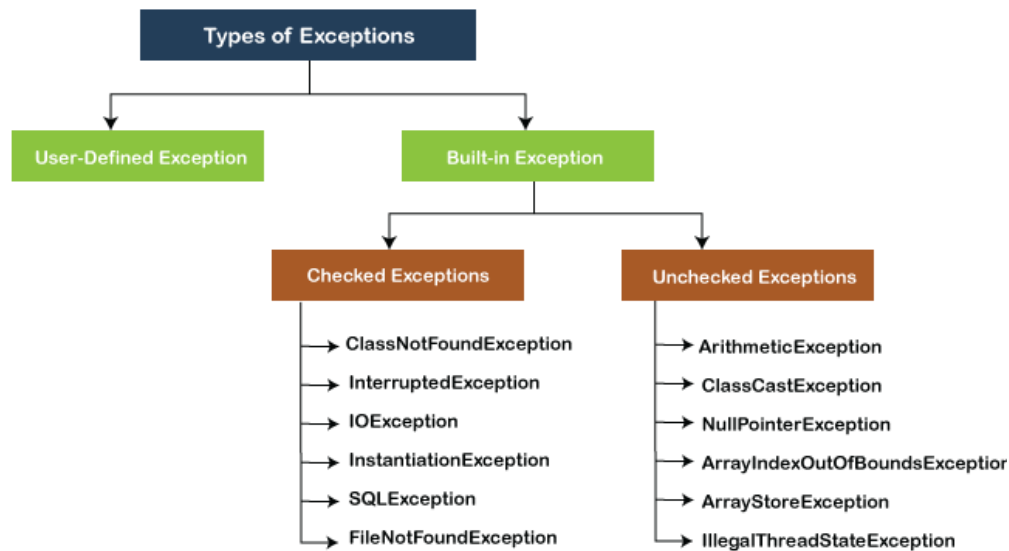
Hasil Output :

Output

```
Index array di luar batas!  
Program selesai dieksekusi.
```

```
=== Code Execution Successful ===
```

Tipe Exception dibagi menjadi 2 bagian, yaitu :



1. Built-in Exception

Built-in Exception adalah exception yang sudah disediakan di dalam Java Standard Library. Exception ini dapat digunakan langsung oleh programmer tanpa perlu mendefinisikannya terlebih dahulu. Biasanya, exception ini muncul akibat kesalahan umum dalam pemrograman. Contoh Built-in Exception yang umum dijumpai:

- **IOException:** Terjadi ketika operasi input atau output gagal atau terinterupsi.
- **ArithmeticException:** Terjadi akibat kesalahan perhitungan aritmatika, seperti pembagian dengan angka nol.
- **ArrayIndexOutOfBoundsException:** Terjadi saat mencoba mengakses indeks array yang tidak tersedia atau di luar batas.
- **ClassNotFoundException:** Terjadi ketika aplikasi mencoba memanggil suatu kelas namun kelas tersebut tidak ditemukan.
- **NullPointerException:** Terjadi ketika program mencoba mengakses atau memanipulasi objek yang bernilai kosong (null).

2. User-Defined Exception

User-Defined Exception adalah kelas exception khusus yang dibuat secara mandiri oleh programmer untuk menangani kasus kesalahan yang tidak dapat diakomodasi hanya dengan built-in exception.

Manfaat User-Defined Exception:

- **Error Handling yang Lebih Fleksibel:** Memungkinkan penyesuaian proses penanganan error agar lebih relevan dengan kasus bisnis yang spesifik.
- **Meningkatkan Readability:** Pesan error menjadi lebih jelas, deskriptif, dan mudah dipahami oleh pengembang lain.
- **Lebih Terstruktur:** Membantu memisahkan berbagai jenis error sehingga kode program menjadi lebih terorganisir.

Bagaimana cara membuat User-Defined Exception?

User-defined exception dibuat dengan mendeklarasikan sebuah **kelas baru** yang merupakan **turunan (subclass)** dari kelas **Exception**. Dengan mewarisi kelas ini, exception yang dibuat dapat dilemparkan (throw) saat kondisi error yang didefinisikan terpenuhi.

Berikut adalah contoh penggunaan **Throws** untuk deklarasi exception di konstruktor/method, dan **Throw** untuk kondisi spesifik yang dapat menyebabkan exception.

```
public class UserDefinedException extends Exception {
    public UserDefinedException(String message) {
        super(message);
    }
}

public class Something {
    public Something(int nilai) throws UserDefinedException {
        if (nilai < 0) {
            throw new UserDefinedException("Nilai tidak boleh negatif!");
        } else {
            System.out.println("Nilai valid: " + nilai);
        }
    }
}
```

Cara Menampilkan Pesan Exception dalam Java

Berikut ini adalah tiga metode utama untuk menampilkan informasi exception dalam Java, yaitu :

1. `getMessage()` → Mengembalikan penyebab error dalam bentuk pesan singkat.
2. `toString()` → Mengembalikan nama exception dan penyebabnya.
3. `printStackTrace()` → Menampilkan pesan `toString()` serta jejak error (stack trace)

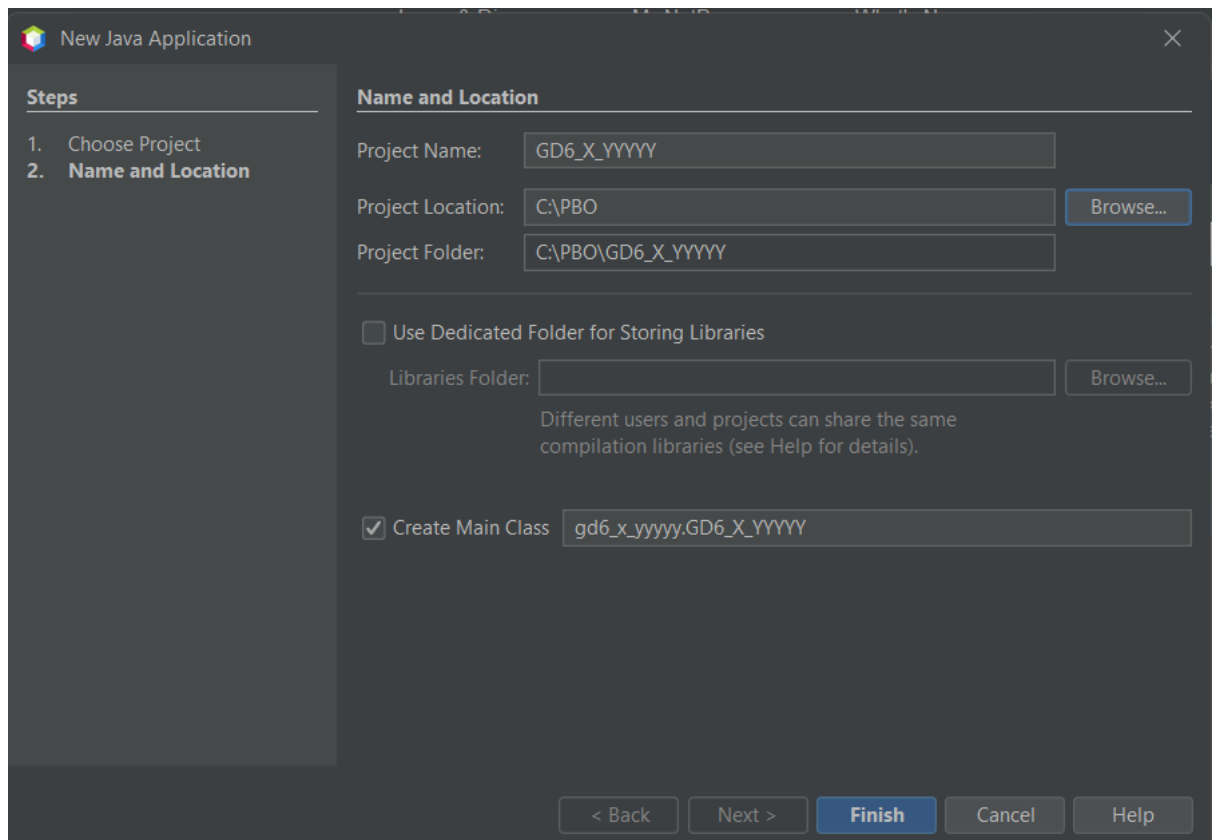


Outputnya dari Program :

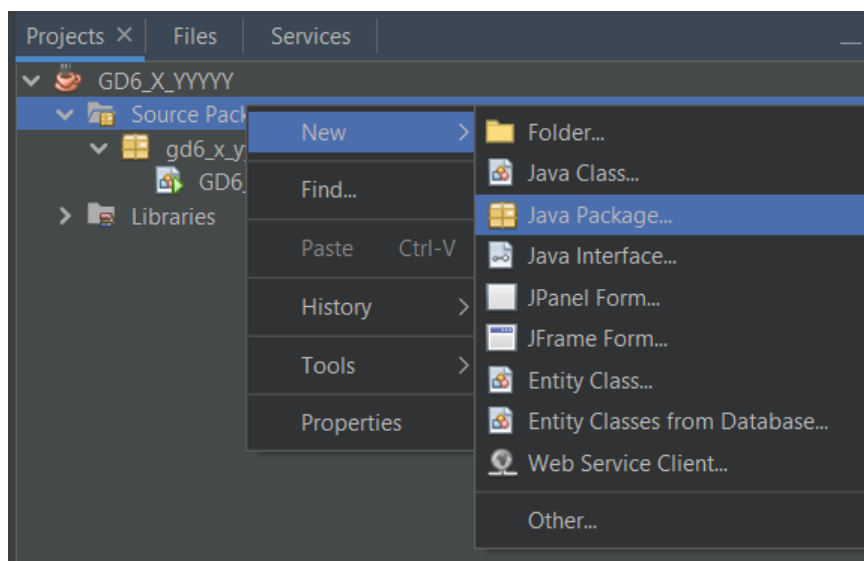
1. `getMessage():` Index 5 out of bounds for length 3
2. `toString():` java.lang.ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 3
3. `printStackTrace():` java.lang.ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 3 at Example.main(Example.java:5)

GUIDED

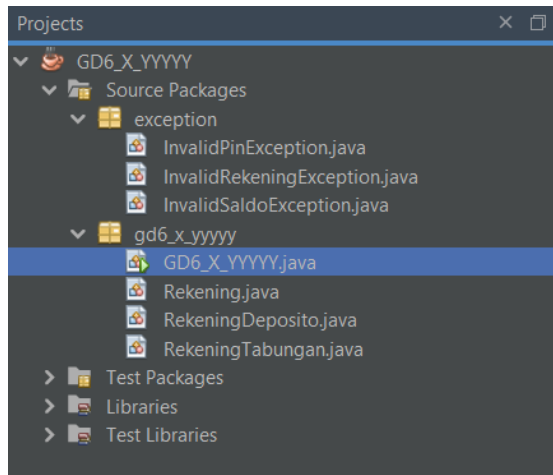
1. Buatlah sebuah project pada netbeans kalian dengan nama **GD6_X_YYYYY** (X = Kelas, YYYYYY = 5 digit terakhir NPM).



2. Buatlah **Package baru bernama exception**. Tujuan dari pembuatan package yang terpisah adalah untuk menjaga kerapihan struktur kode dan memudahkan pemahaman. Jangan lupa untuk melakukan import saat menggunakan kelas dari package lain.



3. Di dalam package exception tersebut, kita akan membuat 3 **user-defined exception**, yaitu **InvalidRekeningException**, **InvalidSaldoException**, dan **InvalidPinException**. Semua kelas tersebut harus meng-extend kelas **Exception**, yang merupakan built-in class di Java.



InvalidSaldoException.java

```
1 package exception;
2
3 /**
4  *
5  * @author Nama Praktikan - NPM
6  */
7 public class InvalidSaldoException extends Exception {
8     // Cara Pertama menggunakan Konstruktor dengan parameter
9     // untuk menetapkan pesan exception.
10    public InvalidSaldoException(String message) {
11        super(message);
12    }
13 }
14
```

Kelebihan InvalidSaldoException adalah fleksibilitas dalam menetapkan pesan error. Pesan error dapat ditentukan langsung melalui konstruktor, sehingga satu exception dapat digunakan untuk berbagai jenis pesan kesalahan.

InvalidRekeningException.java

```
1 package exception;
2
3 /**
4  *
5  * @author Nama Praktikan - NPM
6  */
7 public class InvalidRekeningException extends Exception {
8     // Cara kedua menggunakan konstruktor
9     // dengan pesan exception yang telah ditentukan.
10    public InvalidRekeningException() {
11        super("Nomor rekening harus lebih dari 0!");
12    }
13 }
```

InvalidPinException.java

```
1 package exception;
2
3 /**
4  *
5  * @author Nama Praktikan - NPM
6  */
7 public class InvalidPinException extends Exception {
8     // Cara Ketiga menggunakan method public
9     // dengan pesan exception yang telah ditentukan.
10    public void printError() {
11        System.err.println("PIN harus terdiri dari 6 digit angka valid!");
12    }
13 }
```

Untuk exception `InvalidRekeningException` dan `InvalidPinException`, keduanya sudah membuat pesan error di dalam kelas. Perbedaannya hanyalah cara menampilkan pesan tersebut di blok catch. Jika pesan dibuat di konstruktor, menampilkannya dengan `getMessage`. Jika pesan dibuat di method, menampilkannya dengan memanggil method tersebut.

Tips : Dari 3 contoh cara menampilkan pesan error, silahkan untuk ke depannya menggunakan cara yang paling disukai.

4. Buat kelas induk **Rekening**, ingat untuk meng-import exception yang dibutuhkan

```
1 package gd6_x_yyyyy;
2
3 import exception.InvalidSaldoException;
4 import exception.InvalidRekeningException;
5
6 /**
7  *
8  * @author Nama Praktikan - NPM
9  */
10 public class Rekening {
11     protected double saldo;
12     protected int nomorRekening;
13
14     public Rekening(double saldo, int nomorRekening) throws InvalidSaldoException, InvalidRekeningException {
15         if (saldo < 0) {
16             throw new InvalidSaldoException("Saldo awal tidak boleh negatif!");
17         }
18         if (nomorRekening <= 0) {
19             throw new InvalidRekeningException();
20         }
21         this.saldo = saldo;
22         this.nomorRekening = nomorRekening;
23     }
24
25     public void display() {
26         System.out.println("Nomor Rekening: " + nomorRekening);
27         System.out.println("Saldo: Rp" + saldo);
28     }
29
30     public double getSaldo() {
31         return saldo;
32     }
33 }
```

5. Buat kelas anak yang pertama yaitu **RekeningTabungan**. Dapat dilihat bahwa dalam konstruktor, tetap perlu mendeklarasikan semua exception, termasuk yang sudah dideklarasikan di kelas induk.

```
1 package gd6_x_yyyyy;
2 import exception.InvalidPinException;
3 import exception.InvalidSaldoException;
4 import exception.InvalidRekeningException;
5 /**
6  *
7  * @author Nama Praktikan - NPM
8  */
9 public class RekeningTabungan extends Rekening{
10     private int pin;
11
12     public RekeningTabungan(int pin, double saldo, int nomorRekening) throws InvalidPinException,
13         InvalidSaldoException, InvalidRekeningException {
14         super(saldo, nomorRekening);
15         if (pin < 100000 || pin > 999999) {
16             throw new InvalidPinException();
17         }
18         this.pin = pin;
19     }
20
21     @Override
22     public void display() {
23         super.display();
24         System.out.println("PIN (Terenskripsi): ***" + (pin % 1000));
25     }
26 }
27
```

6. Selanjutnya membuat kelas anak yaitu **RekeningDeposito**.

```
1 package gd6_x_yyyyy;
2 import exception.InvalidSaldoException;
3 import exception.InvalidRekeningException;
4
5 /**
6  *
7  * @author Nama Praktikan - NPM
8  */
9 public class RekeningDeposito extends Rekening{
10     private int jangkaWaktu;
11
12     public RekeningDeposito(int jangkaWaktu, double saldo, int nomorRekening)
13         throws InvalidSaldoException, InvalidRekeningException {
14         super(saldo, nomorRekening);
15         this.jangkaWaktu = jangkaWaktu;
16     }
17
18     @Override
19     public void display() {
20         super.display();
21         System.out.println("Jangka Waktu: " + jangkaWaktu + " Bulan");
22     }
23 }
24
```

7. Buat kelas **Main** untuk menjalankan program. **Ingatlah untuk mengimpor semua user-defined exception yang diperlukan.** Ubah Nama dan NPM Praktikan sesuai dengan identitas masing-masing.

```
1 package gd6_x_yyyyy;
2 import java.io.BufferedReader;
3 import java.io.InputStreamReader;
4 import java.util.Random;
5
6 import exception.InvalidSaldoException;
7 import exception.InvalidRekeningException;
8 import exception.InvalidPinException;
9
10 /**
11  * @author Nama Praktikan - NPM
12  */
13 public class GD6_X_YYYYY {
14
15     public static BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
16
17     public static void main(String[] args) {
18         int nomorRekening, pin, jangkaWaktu;
19         double saldo;
20
21         try {
22             Random random = new Random();
23             int randomNumber = random.nextInt(300);
24
25             System.out.println("TRANSAKSI KE-" + randomNumber);
26
27             System.out.println("\n--- Buka Rekening Deposito ---");
28             System.out.print("Masukkan Saldo Awal: "); saldo = Double.parseDouble(br.readLine());
29             System.out.print("Masukkan Nomor Rekening: "); nomorRekening = Integer.parseInt(br.readLine());
30             System.out.print("Masukkan Jangka Waktu (Bulan): "); jangkaWaktu = Integer.parseInt(br.readLine());
31
32             RekeningDeposito rd = new RekeningDeposito(jangkaWaktu, saldo, nomorRekening);
33
34             System.out.println("\n--- Buka Rekening Tabungan ---");
35             System.out.print("Masukkan Saldo Awal: "); saldo = Double.parseDouble(br.readLine());
36             System.out.print("Masukkan Nomor Rekening: "); nomorRekening = Integer.parseInt(br.readLine());
37             System.out.print("Masukkan PIN (6 Digit): "); pin = Integer.parseInt(br.readLine());
38
39             RekeningTabungan rt = new RekeningTabungan(pin, saldo, nomorRekening);
40
41             System.out.println("\n\n--- Data Rekening Deposito ---");
42             rd.display();
43
44             System.out.println("\n\n--- Data Rekening Tabungan ---");
45             rt.display();
46
47             System.out.println("\n\n-----");
48             System.out.println("Total Aset: Rp" + (rd.getSaldo() + rt.getSaldo()));
49
50         } catch (InvalidSaldoException | InvalidRekeningException e1) {
51             System.err.println(e1.getMessage()); // menampilkan pesan error dari konstruktor exception
52         } catch (InvalidPinException e2) {
53             e2.printStackTrace(); // panggil method printError() dari exception
54         } catch (NumberFormatException e3) {
55             System.err.println("Inputan hanya berupa angka!"); // menangani kesalahan jika input bukan angka
56         } catch (Exception e) {
57             System.err.println(e.toString()); // menampilkan tipe exception dan pesan error
58         } finally {
59             System.out.println("\n\n--- Nama & NPM Praktikan ---"); // isi dengan Nama dan NPM masing-masing
60         }
61     }
62 }
63
64
```

Catatan penting yang harus diingat :

- Penangkapan exception selalu mengikuti prioritas dari blok catch. Jika kode di dalam blok try menyebabkan dua exception sekaligus, yang akan dilempar adalah exception yang memiliki urutan catch lebih awal.

- User-defined exception perlu di-import jika berada di package lain. Built-in exception dari java.lang (misal: NumberFormatException, ArithmeticException) bisa langsung digunakan, sedangkan dari java IO (misal: IOException, FileNotFoundException) harus di-import.
- Jika variabel diassign langsung valuenya di main (tanpa input BufferedReader/Scanner) dan memiliki tipe data yang salah maka exception yang terjadi adalah Runtime Exception. NumberFormatException hanya terjadi jika menggunakan input BufferedReader/Scanner.

Contoh Runtime Exception(tidak perlu disalin, dipahami saja) :

```
try{
    int n = kk;
    System.out.println("n " + n);
} catch (Exception e){
    e.printStackTrace();
}
```

```
run:
java.lang.RuntimeException: Uncompilable source code - Erroneous tree type: kk
    at ugda.UGDA.main(UGDA.java:24)
BUILD SUCCESSFUL (total time: 2 seconds)
```

Berikut adalah hasil run program Guided. Dapat dilihat bahwa blok Finally(yaitu NPM kalian) selalu jalan, saat ada exception ataupun tidak.

Saldo Salah

```
run:
TRANSAKSI KE-155

--- Buka Rekening Deposito ---
Masukkan Saldo Awal: -5
Masukkan Nomor Rekening: 111
Masukkan Jangka Waktu (Bulan): 3
Saldo awal tidak boleh negatif!

--- Nama & NPM Praktikan ---
BUILD SUCCESSFUL (total time: 21 seconds)
```

Pin Salah

```
run:
TRANSAKSI KE-122

--- Buka Rekening Deposito ---
Masukkan Saldo Awal: 100000
Masukkan Nomor Rekening: 12345
Masukkan Jangka Waktu (Bulan): 3

--- Buka Rekening Tabungan ---
Masukkan Saldo Awal: 100000
Masukkan Nomor Rekening: 11223
Masukkan PIN (6 Digit): 123

PIN harus terdiri dari 6 digit angka valid!

--- Nama & NPM Praktikan ---
BUILD SUCCESSFUL (total time: 24 seconds)
```

Rekening Salah

```
run:
TRANSAKSI KE-36

--- Buka Rekening Deposito ---
Masukkan Saldo Awal: 100000
Masukkan Nomor Rekening: 0
Masukkan Jangka Waktu (Bulan): 3

Nomor rekening harus lebih dari 0!

--- Nama & NPM Praktikan ---
BUILD SUCCESSFUL (total time: 21 seconds)
```

Format input salah(hanya boleh angka)

```
run:
TRANSAKSI KE-221

--- Buka Rekening Deposito ---
Masukkan Saldo Awal: 125k

Inputan hanya berupa angka!

--- Nama & NPM Praktikan ---
BUILD SUCCESSFUL (total time: 3 seconds)
```

Output Benar

```
run:
TRANSAKSI KE-252

--- Buka Rekening Deposito ---
Masukkan Saldo Awal: 200000
Masukkan Nomor Rekening: 12345
Masukkan Jangka Waktu (Bulan): 5

--- Buka Rekening Tabungan ---
Masukkan Saldo Awal: 500000
Masukkan Nomor Rekening: 54321
Masukkan PIN (6 Digit): 123456

--- Data Rekening Deposito ---
Nomor Rekening: 12345
Saldo: Rp200000.0
Jangka Waktu: 5 Bulan

--- Data Rekening Tabungan ---
Nomor Rekening: 54321
Saldo: Rp500000.0
PIN (Terenskripsi): ***456

-----
Total Aset: Rp700000.0

--- Nama & NPM Praktikan ---
BUILD SUCCESSFUL (total time: 36 seconds)
```

Format Pengumpulan

Kumpulkan Guided dengan melakukan zip pada project yang telah dibuat, dengan format penamaan dibawah ini:

- GD6_X_YYYYY.zip
- X = Kelas
- YYYYYY = 5 digit NPM

Hint UGD :

- Pelajari cara kerja try-catch-finally dalam menangani exception, serta perbedaan antara throw dan throws beserta cara penggunaannya.
- Pelajari cara membuat User-Defined Exception.
- Pelajari penggunaan ArrayList.

Kontak Asdos

Jika ada yang ingin ditanyakan atau dibahas bisa menghubungi melalui :

- Teams : 220712111@students.uajy.ac.id (Nicholas Raymond Thosimaru), atau
- Whatsapp: 0895622784598.

Sumber Pembelajaran

- <https://www.tpointtech.com/types-of-exception-in-java>
- <https://www.geeksforgeeks.org/throw-throws-java/>
- <https://www.scientecheasy.com/2020/08/exception-handling-in-java.html/>